Les patrons de conception de création

# Simple Factory

Ce n’est pas un patron

# Pizza Store

- Nous avons besoin d'une *pizza store* capable de préparer des pizzas.



- Le client commandera un type de pizza spécifique :
  - Cheese
  - Veggie
  - Greek
  - Pepperoni
  - etc

- Chaque demande de commande concerne un seul type de pizza.

# Pizza Store

- Lors de la commande d'une pizza, nous devons effectuer certaines opérations sur celle-ci :
  - Prepare
  - Bake
  - Cut
  - Box
- Nous savons que toutes les pizzas doivent mettre en œuvre ces comportements. De plus, nous savons que ces comportements ne changeront pas lors de l'exécution (par exemple, le temps de cuisson d'une pizza au fromage ne variera jamais !).

# Parties variables et parties stables

- **Partie variable :**
  - La classe concrète de la pizza à créer.
- **Parties stables :**
  - Préparer la pizza.
  - La cuire.
  - La découper.
  - L'emballer.
- **Objectif :** isoler la création des pizzas tout en conservant le déroulement commun de la commande.

# Pizza Store

```
public Pizza orderPizza(String type) {

        Pizza pizza = new Pizza();

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
}
```

Cette méthode est chargée de créer la pizza.

Elle appelle des méthodes pour la préparation, la cuisson, etc.

La pizza est renvoyée à l'appelant.

- Créer une instance de **`Pizza()`** n'a pas de sens ici, car nous savons qu'il existe différents types de pizzas.

# Pizza Store

```
public Pizza orderPizza(String type) {

        if (type.equals("cheese")) {
                pizza = new CheesePizza();
        } else if (type.equals("greek") {
                pizza = new GreekPizza();
        } else if (type.equals("pepperoni")) {
                pizza = new PepperoniPizza();
        }

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
}
```

Un paramètre indiquant le type

Code qui change

Code qui reste inchangé

# La pression des changements

- La pression monte si on veut appliquer des changements:
- Par exemple rajouter de nouvelles variétés de pizzas:
  - On remarque que tous les concurrents ont ajouté quelques pizzas tendance à leur carte: la pizza au Clam et la pizza végétarienne. On doit évidemment rester compétitif.
  - On va donc ajouter ces pizzas au menu.
  - Par ailleurs, comme on vend peu de pizzas grecques ces derniers temps, on décide de la retirer de la carte.

  - A chaque changement, nous devrions intervenir dans ce code pour mettre à jour l'instruction **`if/else`** (et potentiellement introduire des bugs dans le code existant).

# La pression des changements

```
public Pizza orderPizza(String type) {
        if (type.equals("cheese")) {
                pizza = new CheesePizza();
        } else if (type.equals("greek") {
                pizza = new GreekPizza();
        } else if (type.equals("pepperoni")) {
                pizza = new PepperoniPizza();
        } else if (type.equals("clam") {
                pizza = new ClamPizza();
        } else if (type.equals("veggie") {
                pizza = new VeggiePizza();
        }
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
}
```

Ce code n'est pas fermé à la modification. Si la pizzeria modifie son offre de pizzas, nous devons ouvrir ce code et le modifier.
C'est ce qui varie. Comme la sélection de pizzas évolue au fil du temps, on devra modifier ce code.

Voici ce qui devrait rester inchangé. Dans l'ensemble, les étapes de préparation, de cuisson et d'emballage d'une pizza sont restées les mêmes depuis de très nombreuses années. Nous ne prévoyons donc pas de modification de ce code, mais uniquement des pizzas qu'il traite.

# La pression des changements

- Il est clair que la gestion de la classe concrète à instancier complique considérablement la méthode **`orderPizza()`** et l'empêche d'être fermée à la modification.
- Mais maintenant qu'on sait ce qui varie et ce qui ne varie pas, il est sans doute temps d'encapsuler cet aspect.

# Solution

- Déplacer la création des pizzas dans une classe séparée.

```
public class SimplePizzaFactory {

        public Pizza createPizza(String type) {
                Pizza pizza = null;

                if (type.equals("cheese")) {
                        pizza = new CheesePizza();
                } else if (type.equals("pepperoni")) {
                        pizza = new PepperoniPizza();
                } else if (type.equals("clam")) {
                        pizza = new ClamPizza();
                } else if (type.equals("veggie")) {
                        pizza = new VeggiePizza();
                }
                return pizza;
        }
}
```

## SimplePizzaFactory

- **Avantage** : on dispose d'un endroit unique pour ajouter une nouvelle pizza.

- **Inconvénient** : à chaque changement, nous devons intervenir dans ce code pour ajouter une nouvelle ligne (**mais au moins, dans un seul endroit!**).

# Revisier `PizzaStore`

```
public class PizzaStore {
    SimplePizzaFactory factory;
    public PizzaStore(SimplePizzaFactory factory) {
        this.factory = factory;
    }
    public Pizza orderPizza(String type) {
        Pizza pizza;
        pizza = factory.createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
    }
}
```

Le ***store*** est composé d'une ***factory***

La création de la ***pizza*** est déléguée à la ***factory***

# Simple Factory

- Ce n'est pas un patron officiel, mais il est couramment utilisé.

- Ce n'est pas un mauvais point de départ.

- Lorsque les gens pensent au « Factory », c'est peut-être en réalité à ceci qu'ils pensent.

# Le changement s'opère maintenant...

- **`PizzaStore`** connaît un grand succès et plusieurs branches a travers le monde peuvent exister.
  - Tunis est intéressée.
  - Sfax est intéressée.
  - Et peut-être qu'un jour, ce sera le tour de Monastir…

- Puisque **`PizzaStore`** repose déjà sur une « Simple Factory », cela devrait être facile. Il suffit de créer une **`PizzaStore`** avec une **`SimpleFactory`** différente.

# Le changement s'opère maintenant...

```
// TunisPizzaFactory utilise une logique if/else différente.
TunisPizzaFactory tunisFactory = new TunisPizzaFactory();

// Créez le store de Tunis en utilisant cette Simple Factory.
PizzaStore tunisStore = new PizzaStore(tunisFactory);

// Commander une pizza
tunisStore.order("Veggie");

// Meme demarche pour le store de Sfax
SfaxPizzaFactory sfaxFactory = new SfaxPizzaFactory();
PizzaStore sfaxStore = new PizzaStore(sfaxFactory);
sfaxStore.order("Veggie");
```

# D'autres changements surviennent...

- Sfax apprécie la **`PizzaStore`**, mais souhaite ajouter davantage de fonctionnalités, comme la programmation d'une livraison.

- Sfax tente d'étendre la **`PizzaStore`**...

```
public class SfaxPizzaStore extends PizzaStore {

    public void scheduleDelivery() {
        ...
    }
}
```

# D'autres changements surviennent...

```
SfaxPizzaFactory sfaxFactory = new SfaxPizzaFactory ();
SfaxPizzaStore sfaxStore = new SfaxPizzaStore(sfaxFactory);

// Commander une pizza
sfaxStore.order("Veggie");
sfaxStore.scheduleDelivery();
```

- Voici ce que dit Sfax :
  - Nous n'avons qu'une seule façon de créer des pizzas. Par conséquent, nous n'avons pas besoin de recourir à la composition pour leur création.
  - Nous ne sommes pas satisfaits de devoir créer à la fois notre classe étendue de **`PizzaStore`** et une **`Factory`** dédiée à la création de pizzas. Ces deux classes entretiennent une relation un-à-un. Ne pourrait-on pas les fusionner ?

# Ce que Sfax recherche

- Un framework permettant à Sfax de :
  - Créer des pizzas « Sfax Style »
  - Ajouter des fonctionnalités spécifiques à Sfax.

# Un Framework pour Pizza Store

```
public abstract class PizzaStore {

        abstract Pizza createPizza(String type);

        public Pizza orderPizza(String type) {
                Pizza pizza = createPizza(type);

                pizza.prepare();
                . . .

                return pizza;
        }
}
```

- REMARQUE : Nous utilisons ici l'héritage pour créer les pizzas, et non pas la composition. Par ailleurs, le constructeur de **`PizzaStore`** a été supprimé.

# Un Framework pour Pizza Store

```
public class SfaxPizzaStore extends PizzaStore {
    Pizza createPizza(String type) {
        if (type.equals("cheese"))
                return new SfaxStyleCheesePizza();
        else if (type.equals("veggie"))
                return new SfaxStyleVeggiePizza();
        else if (type.equals("clam"))
                return new SfaxStyleClamPizza();
        else if (type.equals("pepperoni"))
                return new SfaxStylePepperoniPizza();
        else return null;
    }

    void scheduleDelivery() {
        ...
    }
}
```

- La sous-classe définit la manière dont la pizza est préparée... et elle offre également une fonctionnalité spécifique applicable à Sfax.

# Factory Method

```
public abstract class PizzaStore {

        abstract Pizza createPizza(String type);

        public Pizza orderPizza(String type) {
                Pizza pizza = createPizza(type);

                pizza.prepare();
         . . .
                return pizza;
        }
}
```

- La **Factory Method** définit simplement une interface pour la création d'un produit (dans ce cas, un type de pizza). Ce sont les sous-classes qui décident quel produit spécifique créer.

Les patrons de conception de création

# Factory Method

# Introduction au patron Factory Method

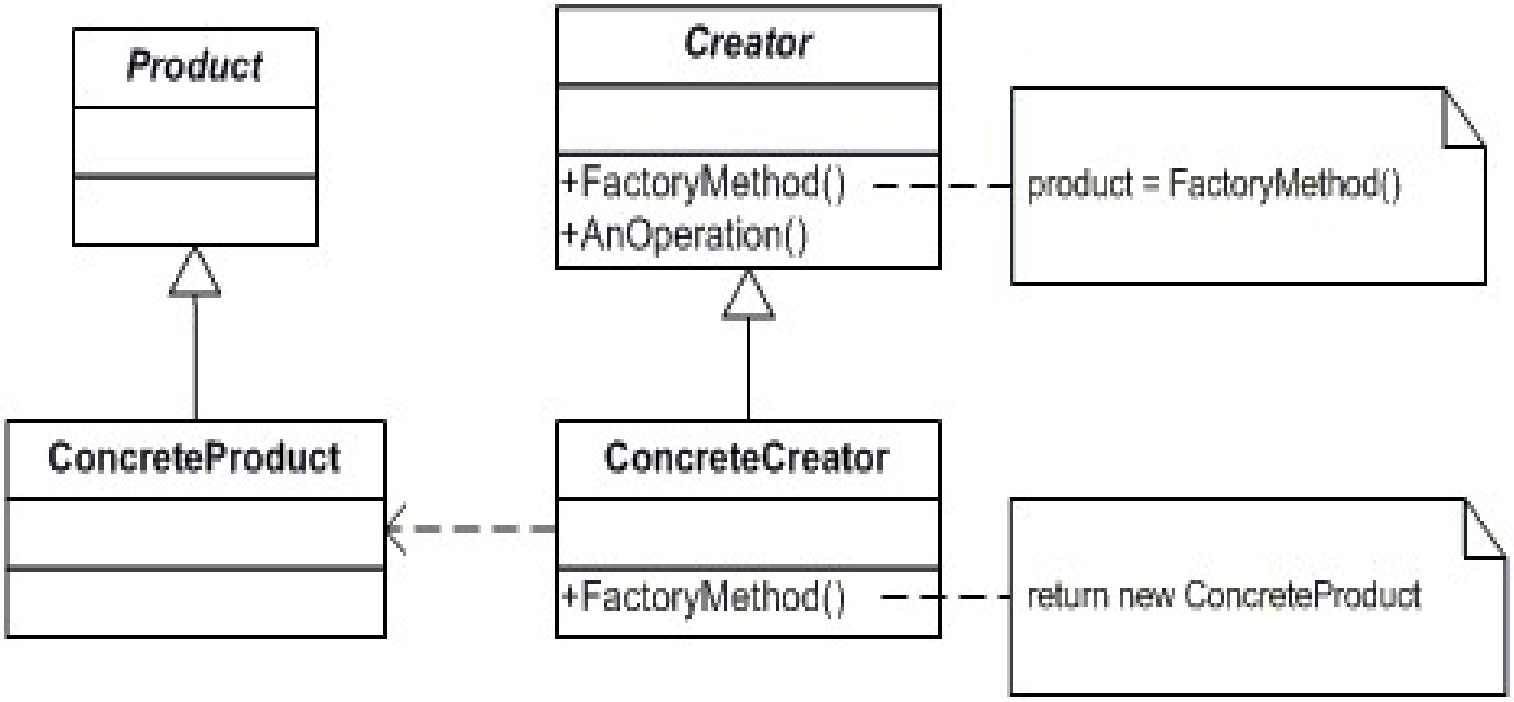
- **Le problème :** créer des objets sans dépendre de leurs classes concrètes.
- Lorsqu'une classe doit créer des objets, elle peut instancier directement leurs classes concrètes à l'aide de **`new`**.

```
Document doc = new DocumentPDF();
```

- Cette approche pose un problème lorsque le type de document à créer peut varier : la classe qui utilise le document dépend directement de **`DocumentPDF`**.
- Si de nouveaux types de documents apparaissent, il peut être nécessaire de modifier cette classe.
- **Question** : comment déléguer la création des objets à des sous-classes, tout en conservant un code indépendant des classes concrètes créées ?
- Le patron Factory Method apporte une réponse à ça.

# Définition selon GoF

- Définit une interface pour la création d'un objet, mais laisse aux sous-classes de décider quelle classe instancier. Le patron **Factory Method** permet à une classe de déléguer l'instanciation aux sous-classes.



# Définition et analogie avec l'exemple de Pizza Store

- Produit – abstraction de l'objet à créer (**`Pizza`**)

- Produit concret – implémentation d'un produit spécifique (**`SfaxStyleCheesePizza`**)

- Créateur – abstraction de l'objet chargé de créer un produit (**`PizzaStore`**). Contient une méthode de factory (factory method).

- Créateur concret – implémentation d'un créateur qui produit un produit concret spécifique (**`SfaxPizzaStore`**)

# Intention du patron “Factory Method”

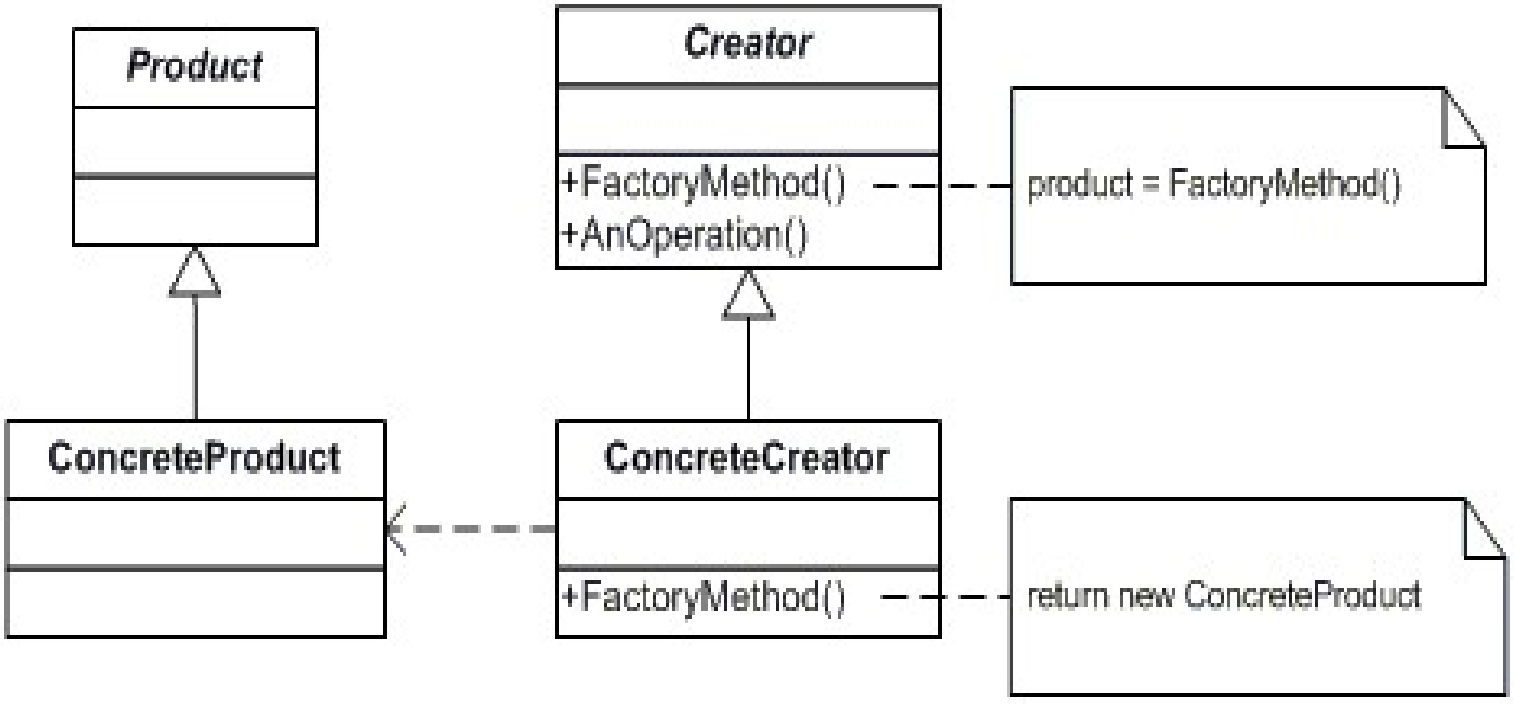
- L'intention de Factory Method est de déléguer la création des objets aux sous-classes afin de séparer la création de l'utilisation.
- Ce patron vise notamment à :
  - Encapsuler la création des objets dans une méthode dédiée.
  - Réduire le couplage entre le code client et les classes concrètes.
  - Permettre aux sous-classes de choisir le type d'objet créé.
  - Faciliter l'évolution du système lorsque de nouveaux produits sont ajoutés.
- **Idée essentielle** : le code métier utilise un objet à travers une interface, tandis que la sous-classe responsable de la création détermine son type concret.

# Circonstances d'application

- Le patron Factory Method est particulièrement adapté dans les situations suivantes :
  - Le type concret de l'objet varie : une classe utilise des objets dont le type dépend de la sous-classe utilisée.
  - Le code doit être extensible : on souhaite ajouter de nouveaux types de produits sans modifier la logique métier existante.
  - Un framework doit être personnalisable : le framework fournit une logique générale, mais laisse les applications qui l'étendent choisir certains objets.
  - La création doit être séparée de l'utilisation : le code qui exploite un objet ne doit pas connaître les détails de sa construction.
- Exemple : une application de logistique réalise les mêmes opérations de livraison, mais utilise des camions pour le transport routier et des bateaux pour le transport maritime.

# Structure générale du patron

- Le patron Factory Method repose sur deux hiérarchies complémentaires :
  - Une hiérarchie de produits, qui définit les objets à créer.
  - Une hiérarchie de créateurs, qui définit la méthode de création et son comportement associé.



# Les participants

- Le patron Factory Method comporte quatre participants principaux.

| Participant | Rôle |
|---|---|
| **`Product`** | Définit l'interface commune des produits. |
| **`ConcreteProduct`** | Implémente l'interface **`Product`**. |
| **`Creator`** | Déclare la méthode fabrique et contient la logique qui utilise le produit. |
| **`ConcreteCreator`** | Redéfinit la méthode fabrique pour fournir un produit concret. |

- A retenir : le **`Creator`** dépend de l'abstraction **`Product`**, et non d'une classe concrète telle que **`ConcreteProductA`** ou **`ConcreteProductB`**.
- Cette dépendance aux abstractions permet de faire évoluer les produits sans réécrire la logique métier du créateur.

# Exemple : une application de logistique

- Considérons une application qui organise des livraisons.
- Elle doit prendre en charge deux modes de transport :
  - Le transport routier, assuré par des camions.
  - Le transport maritime, assuré par des bateaux.
- Les deux types de transport possèdent une opération commune : **`livrer()`**.
- L'application doit conserver la même logique générale de livraison, quel que soit le mode de transport





**Transport routier**
Produit concret : **`Truck`**
La livraison s'effectue par la route.

**Transport maritime**
Produit concret : **`Ship`**
La livraison s'effectue par voie maritime.

# Exemple : une application de logistique

- Correspondance avec les participants du patron :
  - **`Transport : Product`**
  - **`Truck`** et **`Ship : ConcreteProduct`**
  - **`Logistics : Creator`**
  - **`RoadLogistics`** et **`SeaLogistics : ConcreteCreator`**

# Fonctionnement du patron

- Le fonctionnement se déroule en quatre étapes :
  1. Le code client utilise un créateur concret, par exemple **`RoadLogistics`**.
  2. Il appelle une opération définie dans la classe **`Creator`** par exemple **`planDelivery()`**.
  3. Cette opération appelle la méthode **`factoryMethod()`**.
  4. La méthode redéfinie dans RoadLogistics crée un **`Truck`** et le retourne sous le type abstrait **`Transport`**.
- Le créateur utilise ensuite le produit retourné sans avoir besoin de connaître sa classe concrète.

# Implémentation en Java : le produit

- L'interface **`Transport`** définit le comportement commun à tous les moyens de transport.

```
public interface Transport {
    void livrer();
}
```

# Implémentation en Java : le produit

- Les produits concrets implémentent l'interface **`Transport`**.

```
public class Truck implements Transport {
  public void livrer() {
    System.out.println("Livraison par la route.");
  }
}

public class Ship implements Transport {
  public void livrer() {
    System.out.println("Livraison par voie maritime.");
  }
}
```

- Les classes **`Truck`** et **`Ship`** réalisent la même opération, mais chacune fournit sa propre implémentation.

# Implémentation en Java : le créateur

- La classe abstraite **`Logistics`** définit la méthode fabrique **`createTransport()`** et une opération générale **`planDelivery()`**.

```
public abstract class Logistics {

    protected abstract Transport createTransport();

    public void planDelivery() {
        Transport t = createTransport();
        t.livrer();
    }
}
```

# Implémentation en Java : le créateur

- **`createTransport()`** est la méthode fabrique (Factory Method).
- Son type de retour est l'interface **`Transport`**.
- **`planDelivery()`** représente la logique métier commune.
- La classe **`Logistics`** ne connaît ni **`Truck`** ni **`Ship`**.
- La création du produit est déléguée aux sous-classes, tandis que l'organisation de la livraison reste définie dans la classe de base.

# Implémentation en Java : les créateurs concrets

- Chaque sous-classe redéfinit la méthode fabrique pour déterminer le type de transport à utiliser.

```
public class RoadLogistics extends Logistics {
    @Override
    protected Transport createTransport() {
        return new Truck();
    }
}

public class SeaLogistics extends Logistics {
    @Override
    protected Transport createTransport() {
        return new Ship();
    }
}
```

# Implémentation en Java : les créateurs concrets

- **`RoadLogistics`** crée des objets **`Truck`**.
- **`SeaLogistics`** crée des objets **`Ship`**.
- La méthode **`planDelivery()`** n'a pas besoin d'être redéfinie : elle utilise automatiquement le produit fourni par la méthode fabrique de la sous-classe.

# Utilisation du patron

- Le code client choisit le créateur concret à utiliser, puis appelle l'opération commune.

```
public class Main {
    public static void main(String[] args) {

        Logistics logistics = new RoadLogistics();
        logistics.planDelivery();

        logistics = new SeaLogistics();
        logistics.planDelivery();
    }
}
```

Résultat :

```
Livraison par la route.
Livraison par voie maritime.
```

# Utilisation du patron

- Le client connaît les classes concrètes des créateurs dans cet exemple, mais il manipule leurs objets à travers le type abstrait Logistics. Il n'a pas besoin de créer lui-même les objets **`Truck`** ou **`Ship`**.
- Le choix du créateur concret peut être effectué à partir d'une configuration, d'un environnement d'exécution ou d'un autre mécanisme de sélection.

# Avantages du patron

- Le patron Factory Method apporte plusieurs avantages.
  - Réduction du couplage : le créateur manipule l'interface **`Product`** plutôt que les classes concrètes.
  - Extensibilité : il est possible d'ajouter un nouveau type de produit en créant un produit concret et un créateur concret adaptés.
  - Réutilisation de la logique métier : les sous-classes héritent des opérations communes du créateur.
  - Respect du principe ouvert/fermé : de nouvelles variantes peuvent être ajoutées sans modifier la logique métier existante, lorsque la conception a prévu ce point d'extension.
- Par exemple, pour ajouter un transport ferroviaire, on pourrait créer **`Train`** et **`RailLogistics`** sans modifier le code de **`Logistics`**.

# Limites du patron

- Factory Method présente également certaines limites :
  - Multiplication des classes : chaque variante peut nécessiter un produit concret et un créateur concret.
  - Complexité supplémentaire : une simple création d'objet peut devenir inutilement complexe si aucune variation réelle n'est nécessaire.
  - Dépendance à l'héritage : la variation du produit repose généralement sur la redéfinition d'une méthode dans une sous-classe.
- Recommandation : utiliser ce patron lorsque le choix du type concret fait naturellement partie d'une hiérarchie de créateurs, et non pour remplacer systématiquement chaque appel à **`new`**.

# Factory Method et Simple Factory : quelle différence ?

- Ces deux approches sont souvent confondues.

| Simple Factory | Factory Method GoF |
|---|---|
| Une méthode choisit le produit à créer. | Une méthode fabrique est redéfinie dans les sous-classes du créateur. |
| Le choix repose souvent sur un paramètre et un if ou switch. | Le choix dépend généralement du créateur concret utilisé. |
| N'exige pas de hiérarchie de créateurs. | Repose sur une hiérarchie de créateurs. |

# Factory Method et Simple Factory : quelle différence ?

- Exemple de Simple Factory :

```
static Transport creerTransport(String type) {
    if (type.equals("route"))
        return new Truck();
    else
        return new Ship();
}
```

- Cette méthode centralise la création, mais elle ne constitue pas, à elle seule, le patron Factory Method du GoF. Le patron GoF se distingue par la délégation de la création à une méthode redéfinie dans les sous-classes.